

Dynamic Programming Study Guide

Dynamic Programming Study Guide

1. Overview

Dynamic Programming (DP) is a technique for solving problems with **overlapping subproblems** and **optimal substructure**. Instead of recomputing results, DP stores intermediate values to achieve efficient solutions.

DP problems often fall into two categories:

- **Top-Down (Memoization):** Recursion + caching
- **Bottom-Up (Tabulation):** Iterative table-filling

Understanding DP requires recognizing patterns rather than memorizing solutions.

2. How to Identify a DP Problem

A problem is likely DP if:

- The solution involves **making decisions** at each step.
- The problem can be broken into **smaller subproblems**.
- Subproblems **repeat**.
- You can define a **state** that represents a partial solution.

Common signals:

- "Maximum", "minimum", "longest", "count ways"
 - Problems involving **choices** (take / skip)
 - Problems involving **sequences, grids, or combinatorics**
-

3. Core DP Patterns

3.1 1D DP

Used when the state depends on a single index. Examples:

- Fibonacci numbers
- Climbing stairs
- House robber

Template:

```
dp = [0] * (n+1)
for i in range(1, n+1):
    dp[i] = ... # transition
return dp[n]
```

3.2 2D DP

Used when the state depends on two indices. Examples:

- Longest Common Subsequence
- Edit Distance
- Knapsack

Template:

```
dp = [[0]*(m+1) for _ in range(n+1)]
for i in range(n):
    for j in range(m):
        dp[i][j] = ...
return dp[n-1][m-1]
```

3.3 Knapsack Pattern

Used when choosing items under constraints. Examples:

- 0/1 Knapsack
- Partition Equal Subset Sum
- Target Sum

Transition:

```
dp[i][w] = max(dp[i-1][w], dp[i-1][w-weight[i]] + value[i])
```

3.4 Subsequence Pattern

Used for string/array subsequence problems. Examples:

- LIS
 - LCS
 - Palindromic subsequences
-

3.5 Grid DP

Used for path-counting or path-cost problems. Examples:

- Unique Paths

- Minimum Path Sum
-

4. Common DP Transitions

- **Take / Skip:** $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
 - **Min/Max:** $dp[i][j] = \min(dp[i-1][j], dp[i][j-1]) + \text{grid}[i][j]$
 - **Count ways:** $dp[i] += dp[i - \text{coin}]$
 - **Boolean:** $dp[i][j] = dp[i-1][j] \text{ or } dp[i-1][j - \text{nums}[i]]$
-

5. Example Problem: Climbing Stairs

Problem: You can climb 1 or 2 steps. How many ways to reach step n ?

State: $dp[i]$ = ways to reach step i

Transition: $dp[i] = dp[i-1] + dp[i-2]$

Solution:

```
def climbStairs(n):
    if n <= 2: return n
    a, b = 1, 2
    for _ in range(3, n+1):
        a, b = b, a+b
    return b
```

6. Common Pitfalls

- Using recursion without memoization
 - Incorrect base cases
 - Misdefining the DP state
 - Iterating in the wrong order
 - Using too much memory (optimize with rolling arrays)
-

7. Practice Roadmap

Beginner

- Fibonacci
- Climbing Stairs
- House Robber

- Min Cost Climbing Stairs

Intermediate

- Coin Change
- Longest Increasing Subsequence
- Unique Paths
- Partition Equal Subset Sum

Advanced

- Edit Distance
- Interleaving String
- Burst Balloons
- Word Break

8. Key Takeaways

- DP is about **states**, **transitions**, and **optimal substructure**.
- Most DP problems follow a small number of patterns.
- Practice recognizing patterns instead of memorizing solutions.

End of document.